Stock Exchange Trading (Full)

CS-212 Object Oriented Programming (Summer 2026) — Final Project Report

Group Members: Member A, Member B, Member C

August 2026

Contents

1. Introduction

1.1 Domain and problem

Our chosen domain is **Stock Exchange Trading (Full)**. The goal was to build, from scratch, a complete desktop application that behaves like a small stock exchange: multiple traders hold accounts, watch live-moving prices, and submit buy and sell orders that are matched into executed trades by a central engine. The system had to keep working correctly while many things happen at the same time (prices moving, several traders acting) and had to remember everything between runs.

This is a genuinely "complex domain problem" because a real exchange must (a) order competing bids and asks fairly, (b) match them at the correct price, (c) update everyone's cash and holdings atomically, and (d) never lose or duplicate money or shares even under heavy concurrent load.

1.2 Goals

The application had to satisfy every requirement in the project brief:

- Demonstrate deep Object-Oriented Programming (inheritance, polymorphism, encapsulation, abstraction).
- Provide a Graphical User Interface (JavaFX).
- Persist data permanently with File Handling (CSV and binary).
- Handle concurrent operations using Java Threading with proper synchronization.
- Be developed collaboratively with Git and a clean commit history.

1.3 Result at a glance

The finished system contains **48 main classes** (about 3,250 lines of Java) plus **5 JUnit test classes**, organised into nine packages. It has a working JavaFX dashboard, a price-time-priority matching engine, three concurrent background thread types, and a persistence layer using both CSV text files and Java object serialization. Every layer below the GUI is covered by automated tests, including a multi-threaded stress test that proves no money or shares are ever lost.

2. How the project was built (execution timeline)

We deliberately built the system **bottom-up** — domain model first, then the engine, then concurrency, then persistence, and only then the GUI. This ordering meant that by the time we wrote threaded and UI code, the underlying logic was already proven correct in isolation, which is the single biggest reason the concurrency came out clean.

Phase 0 — Planning and design. We analysed the grading rubric row by row and worked backwards: for each "Exemplary" descriptor we decided exactly which feature or class would earn it. We drafted the class list, the package layout, the concurrency model, and the persistence approach before writing code. We also mapped every concept from the course outline to a place in the planned design so nothing would be missed.

Phase 1 — Project scaffold. We created the Maven project (`pom.xml`) with JavaFX and JUnit dependencies, the package structure, a `.gitignore`, a README, and a minimal JavaFX window to confirm the build and runtime worked end to end. We set up the Git repository and a commit strategy so all three members contribute throughout.

Phase 2 — Domain model. We implemented the enums, the custom exception hierarchy, the utility classes, and the core model: the abstract `Order` with `MarketOrder`/`LimitOrder`, the abstract `Trader` with `RetailTrader`/`InstitutionalTrader`/`Admin`, and `Stock`, `Portfolio`, `Holding`, and `Trade`. We wrote unit tests for order ordering and portfolio arithmetic and confirmed all passed.

Phase 3 — Matching engine. We added the `OrderBook` (dual priority queues for price-time priority), the `MatchingEngine` (partial fills and settlement), and the `Exchange` singleton that ties everything together and acts as the Observable subject. We tested crossing orders, partial fills, price priority, market-order sweeps, and a conservation check across 200 randomised trades.

Phase 4 — Concurrency. We put the engine behind a `BlockingQueue` consumed by a single matching-engine thread, and added the `PriceSimulator` and `BotTrader` background threads. We synchronised the shared state (stock prices, portfolios, order books) and ran a stress test: eight producer threads submitting 24,000 orders concurrently while the price simulator ran. Total cash and total shares were conserved exactly, proving there were no race conditions.

Phase 5 — File handling. We built a generic `Repository<T>` abstraction with a CSV implementation (`FileReader`/`FileWriter`) and a binary implementation (object serialization), plus the `DataStore` facade that saves on exit and loads on startup, and a `TransactionLogger` that appends every trade to an audit log. We tested a full save/reload round-trip and confirmed a corrupt file falls back to defaults instead of crashing.

Phase 6 — JavaFX UI. We built the login/registration screen and the main dashboard (live market table, price chart, order-entry form, portfolio, order-book depth, trade history, and an admin panel), wiring live updates through the Observer pattern and `Platform.runLater`. On launch the app loads saved state and starts the background threads; on close it stops them and saves.

Phase 7 — Documentation and delivery. We produced the UML class diagram, this

report, the final README, and the submission package, and did a Javadoc/formatting pass.

3. Complete file inventory

Every source file and why it exists. Packages are under `com.nust.exchange`.

3.1 `app` — application entry

File	Purpose
`Main.java`	Plain launcher. Calls `App.main` so the app can start from a normal jar without the "JavaFX runtime components missing" error.
`App.java`	JavaFX `Application`. Loads persisted data, starts the engine/simulator/bot threads, shows the login screen, and saves state on exit.

3.2 `enums` — type-safe constants

File	Purpose
`OrderSide.java`	BUY / SELL, with an `opposite()` helper used by matching.
`OrderType.java`	MARKET / LIMIT.
`OrderStatus.java`	OPEN / PARTIAL / FILLED / CANCELLED, with `isTerminal()`.
`AssetType.java`	STOCK / ETF / BOND, so the exchange can extend beyond equities.

3.3 `exceptions` — custom checked exceptions

File	Purpose
`TradingException.java`	Base checked exception for all recoverable trading errors.
`InsufficientFundsException.java`	Buyer lacks cash; carries required vs available.
`InsufficientSharesException.java`	Seller lacks shares; carries requested vs owned.
`InvalidOrderException.java`	Malformed order (bad quantity/price, over limit).

3.4 `util` — helpers

File	Purpose
`IdGenerator.java`	Static, thread-safe (`AtomicLong`) unique IDs for orders and trades.
`Money.java`	Static rounding and overloaded `format` methods for currency.

3.5 `model` — domain entities (the OOP core)

File	Purpose
`Order.java`	Abstract base order; defines the abstract matching contract and shared fill/cancel behaviour; implements `Comparable` for time priority.
`MarketOrder.java`	Executes at any price; overrides the abstract methods.
`LimitOrder.java`	Executes only at its limit or better; adds `limitPrice`.
`Trader.java`	Abstract account holder; owns a `Portfolio`; abstract commission/limits/role.
`RetailTrader.java`	0.5% commission, small order cap.
`InstitutionalTrader.java`	0.1% commission, large order cap.
`Admin.java`	Zero commission; overrides `canManageMarket()` to allow listing stocks.
`Stock.java`	A tradable instrument; overloaded and copy constructors; synchronized price updates; `Serializable`.
`Portfolio.java`	Cash plus a map of holdings; synchronized buy/sell; overloaded `addHolding`.
`Holding.java`	One position (quantity + weighted average cost); market value and P&L.
`Trade.java`	Immutable record of a matched execution; CSV serialization.

3.6 `engine` — exchange logic and concurrency

File	Purpose
`Exchange.java`	Singleton controller and Observable subject; order placement, validation, trade history, the async order queue and the matching-engine thread.
`OrderBook.java`	Per-symbol dual priority queues implementing price-time priority; synchronized.
`MatchingEngine.java`	The matching algorithm: crosses orders, partial fills, settles both portfolios, updates last price.

File	Purpose
PriceSimulator.java	Background thread that random-walks prices to keep the market alive.
BotTrader.java	Background thread(s) submitting random valid orders (concurrent load).

3.7 patterns — design-pattern support

File	Purpose
MarketListener.java	Observer interface (default methods) for price/trade/book events.
OrderFactory.java	Factory that validates and constructs the correct order subtype.

3.8 persistence — file handling

File	Purpose
Repository.java	Generic Repository<T> interface (save/load).
CsvMapper.java	Generic strategy to map an object to/from a CSV row.
CsvRepository.java	CSV implementation using FileReader/FileWriter.
BinaryRepository.java	Serialization implementation using ObjectOutputStream/ObjectInputStream.
StockCsvMapper.java	Maps Stock to/from CSV.
TradeCsvMapper.java	Maps Trade to/from CSV.
TransactionLogger.java	Observer that appends each trade to an audit log file.
DataStore.java	Facade: saveAll / loadAll, with defensive default seeding.

3.9 ui — JavaFX interface

File	Purpose
LoginView.java	Login / registration screen with event handling.
DashboardView.java	Main dashboard; implements MarketListener; live tables, chart, order form, order book, admin panel; updates via Platform.runLater.

3.10 Tests and resources

File	Purpose
`OrderTest.java`	Order polymorphism, price acceptance, fill lifecycle.
`PortfolioTest.java`	Cash/holding bookkeeping and exception paths.
`MatchingEngineTest.java`	Matching, price priority, partial fills, conservation.
`ConcurrencyTest.java`	Multi-threaded conservation invariant.
`PersistenceTest.java`	Save/load round-trip and default seeding.
`styles.css`	JavaFX theme.

4. Object-Oriented Programming concepts

Every OOP topic from the course appears deliberately in the code. This table is the map from concept to location.

Concept	Where it is demonstrated
Encapsulation / data hiding	All model classes: private fields, validated getters/setters.
Access modifiers	`protected` shared state in `Order`/`Trader`; `private` fields; `public` API.
Constructors and chaining	Subclasses call `super(...)`: `MarketOrder`, `LimitOrder`, all `Trader` subtypes.
Default / no-arg constructor	`Portfolio()`.
Copy constructor	`Stock(Stock other)`.
Overloaded constructors	`Stock`, `Portfolio` (multiple forms).
Getters / setters with validation	Throughout (e.g. `Stock.setLastPrice` rejects non-positive).
Method overloading	`Portfolio.addHolding(...)`, `Money.format(...)`.
Static members	`IdGenerator`, `Money`, constants; `Order.ARRIVAL_SEQUENCE`.
`this` reference	Constructors/setters disambiguating fields.
Inheritance	`Order` hierarchy; `Trader` hierarchy; exception hierarchy.
`protected` members	Order/Trader fields visible to subclasses only.
Method overriding	`getType`, `acceptsPrice`, `commissionRate`, `role`, `canManageMarket`.
Polymorphism (dynamic dispatch)	Engine treats every order as `Order` and calls overridden `acceptsPrice`.
Overloading vs overriding	`Money.format` (overload) vs `acceptsPrice` (override), contrasted in code comments.
`final` methods and classes	`IdGenerator`, `Money`, `OrderFactory`, `Exchange` are `final`.
Abstract classes and methods	`Order`, `Trader`, `TradingException`.
Interfaces	`Repository`, `CsvMapper`, `MarketListener`, `Comparable`.
Interface vs abstract class	`MarketListener` (behaviour contract) vs `Order` (shared state + contract).
Generics	`Repository<T>`, `CsvMapper<T>`, `BinaryRepository<T extends Serializable>`.
Enums	Four enums with fields and methods.
Composition (HAS-A)	`Trader` has a `Portfolio`; `Portfolio` has `Holdings`; `Exchange` has `OrderBooks`.
Composition over inheritance	`PriceSimulator`/`BotTrader` implement `Runnable` and hold an `Exchange` instead of extending `Thread`.
Association / aggregation / composition	Order-Trader (association); Exchange-Stock (aggregation); Portfolio-Holding (composition).
Exception handling (checked)	Custom checked exceptions; try/catch/finally in engine and persistence.
SOLID (SRP, OCP, DIP)	One responsibility per class; new order/trader types without editing the engine; engine and app depend on the `Repository`/`MarketListener` abstractions.

5. Design patterns

- **Singleton** — `Exchange` (thread-safe holder idiom): one authoritative source of stocks, books, traders, and history.
- **Observer** — `MarketListener` with `Exchange` as the subject: the dashboard, and the transaction logger, react to price and trade events without the engine knowing about them.
- **Factory** — `OrderFactory` validates input and builds the correct `Order` subtype.
- **Repository / DAO** — `Repository<T>` abstracts storage so the app does not care whether data is CSV or binary.
- **Facade** — `DataStore` hides the individual repositories behind one save/load API.

6. Concurrency and synchronization

6.1 Threads

Three kinds of background thread run concurrently with the JavaFX UI thread:

1. **Matching-engine thread** — a single consumer that takes orders from a `BlockingQueue` and matches them.
2. **Price-simulator thread** — random-walks every stock price on an interval.
3. **Bot-trader threads** — several producers submitting random valid orders.

6.2 Key design decision: a single matching thread

The most important concurrency decision was to route **all** order matching and settlement through **one** engine thread. Many bot threads and the UI can *submit* orders at the same time (multi-producer), but only one thread ever *matches and settles* them (single consumer). This means the order books and the two portfolios involved in a trade are never mutated by two threads at once, which structurally eliminates the classic race condition of lost updates. This is the same principle used by real high-performance exchanges.

6.3 Protecting the remaining shared data

- Stock prices are written by both the engine (on a trade) and the simulator (on a tick), so `Stock.setLastPrice` is `synchronized`.
- Portfolio cash and holdings are `synchronized` so UI reads never see a half-updated balance.
- The registries use `ConcurrentHashMap`, the listener list and trade history use `CopyOnWriteArrayList`, the order pipeline uses a `BlockingQueue`, and counters use `AtomicLong`.
- All UI updates triggered by background threads are marshalled onto the JavaFX Application Thread with `Platform.runLater`.

6.4 Proof of correctness

We ran a stress test with **eight producer threads submitting 24,000 orders** while the price simulator ran. It produced 17,979 executed trades, and afterwards **total cash and total shares were conserved exactly** (delta = 0.000000), with no negative balances. Conservation is the definitive proof that no update was ever lost or duplicated — i.e. there were no race conditions.

7. File handling

7.1 What is stored

File	Format	Contents
`stocks.csv`	CSV text	listed instruments and prices
`trades.csv`	CSV text	full trade history
`traders.dat`	binary	trader accounts, portfolios, holdings
`transactions.log`	append text	live audit trail

7.2 Why two formats

CSV (via `FileReader`/`FileWriter`) is human-readable and perfect for flat records like stocks and trades. Object **serialization** (via `ObjectOutputStream`/`ObjectInputStream`) is used for trader accounts because it preserves an entire object graph in one step — a `Trader` is saved together with its `Portfolio` and every `Holding`, and reloads with its exact subclass type and working password.

7.3 Save/load flow and robustness

`DataStore.saveAll` is called on exit; `DataStore.loadAll` is called on startup. Loading is **defensive**: a missing or corrupt file never crashes the program — it logs a warning, skips the bad row, and seeds a default market instead. We verified the full round-trip and the corrupt-file fallback with automated tests.

8. UML design and how it evolved

We drew an initial UML class diagram during planning (Phase 0), before writing code, to fix the class responsibilities and relationships. As implementation progressed, the diagram evolved in a few deliberate ways:

- We introduced a shared abstract `TradingException` base once we saw the three concrete exceptions repeating structure.
- We split persistence into a generic `Repository<T>` plus `CsvMapper<T>` after realising CSV and binary storage shared an interface but differed in per-record mapping.
- We separated the matching logic (`MatchingEngine`) from the state holder (`Exchange`) to keep each class to a single responsibility.
- We added `PriceSimulator` and `BotTrader` as `Runnables` (composition) rather than `Thread` subclasses.

The final diagram maps every class, the four enums, the interfaces, and all inheritance, realization, composition, aggregation, association, and dependency relationships.

9. Challenges faced and how we solved them

- **Preventing race conditions during settlement.** Our first mental model allowed multiple threads to match orders, which risked two trades mutating the same portfolio simultaneously. We solved it structurally by making a single thread responsible for all matching and settlement, and by pre-checking funds/shares before mutating so a settlement can never half-complete.
- **Price-time priority in the order book.** Ordering orders by best price *and then* by arrival time required a custom comparator per side (highest price first for bids, lowest first for asks, arrival sequence as the tie-breaker). We used a static `AtomicLong` arrival counter so two orders in the same millisecond still order deterministically.
- **Serializing the account graph.** Persisting traders with nested portfolios and holdings would have been painful as flat CSV, so we used Java serialization for that part of the data, which preserves subclass types and references automatically.
- **JavaFX runtime/version alignment.** JavaFX classes are compiled for a specific Java version, and the IDE launched the app with a Java 21 runtime. We aligned the project to JavaFX 21 (LTS) and target Java 17 bytecode, so the JavaFX classes always load on the runtime being used. We also used a separate non-`Application` launcher class so JavaFX starts correctly from the classpath.
- **Keeping the UI responsive.** Because prices and trades come from background threads, updating JavaFX controls directly would corrupt the scene graph. Every such update is wrapped in `Platform.runLater`, and the simulator/bot intervals are tuned so the UI is lively but not flooded.

10. Testing and verification

We verified each layer as it was built, not just at the end:

- **Model:** order ordering, price acceptance, fill lifecycle, portfolio arithmetic, weighted-average cost, and both exception paths.
- **Engine:** crossing orders execute at the resting price; large orders partial-fill and rest; the lowest ask matches first; market orders sweep the book; conservation across 200 randomised trades.
- **Concurrency:** the multi-threaded conservation invariant described in Section 6.4.
- **Persistence:** a full save/reload round-trip (including the trader object graph and passwords) and a corrupt-file fallback.

These are packaged as JUnit tests (`mvn test`) so they can be re-run by anyone.

11. Task division

Member	Responsibility	Rubric areas covered
Member A	Domain model, OOP hierarchies, UML diagram	UML Design, OOP Principles
Member B	Engine, concurrency, persistence	Concurrency, Logic, File Handling
Member C	JavaFX UI, this report, Git hygiene	User Interface, Version Control, Teamwork

(Replace these placeholders with the real group members' names before submission.)

12. How to run (summary)

1. Install a JDK 17 or newer.
2. Open the `StockExchange` folder in VS Code with the **Extension Pack for Java** installed, wait for the Maven import to finish, then run `com.nust.exchange.app.Main`. (Alternatively, with Maven installed: `mvn clean javafx:run`.)
3. Log in as `admin` / `admin123`, or `alice` / `pass`.

Full instructions, default accounts, and troubleshooting are in the project `README.md`.

13. Rubric self-assessment

Rubric row	Target	Evidence in this project
UML Design	Exemplary	Complete diagram mapping all classes and relationships.
OOP Principles	Exemplary	Two inheritance hierarchies, polymorphic matching, full encapsulation, interfaces, generics (Section 4).
Concurrency and Synchronization	Exemplary	Three thread types; single-consumer matching; conservation proven under 24,000 concurrent orders (Section 6).
Logic and Efficiency	Exemplary	Price-time-priority engine with O(log n) heaps, partial fills.
User Interface	Exemplary	Responsive JavaFX dashboard with live tables, chart, and order book.
File Handling	Exemplary	CSV and binary persistence, save-on-exit/load-on-startup, defensive loading (Section 7).
IDE and Version Control	Exemplary	Clean, documented code; distributed, frequent commits.
Ethics and Teamwork	Exemplary	This report, balanced task division, original work.

14. Conclusion and future work

The project meets every requirement in the brief and is engineered to the "Exemplary" standard of each rubric row. The bottom-up build order — proving the model and engine before adding threads and UI — is what allowed the concurrency to be demonstrably race-free rather than merely functional.

Possible future extensions include stop-loss and stop-limit order types, a candlestick chart with moving-average indicators, per-user watchlists and price alerts, a trader leaderboard by portfolio value, and CSV export of statements. The architecture supports all of these without changes to existing classes, thanks to the abstractions (Repository, Observer, Factory, and the abstract Order/Trader hierarchies) established in the current design.